Web Tabanlı Sosyal Kütüphane Platformu

Farahnozkhon Dilovarovna MASUMOVA
Kocaeli Üniversitesi
Bilgisayar Mühendisliği Bölümü
Yazılım Laboratuvarı I - Proje II Raporu

***Özet*—Bu proje kapsamında, kullanıcıların kişisel kitap ve film kütüphanelerini oluşturabildiği, içerikleri puanlayabildiği, yorumlayabildiği ve sosyal akış üzerinden diğer kullanıcılarla etkileşim kurabildiği web tabanlı bir sosyal kütüphane platformu geliştirilmiştir. Sistem, *backend* tarafında Django 5 tabanlı bir REST API, *frontend* tarafında ise React ve Vite ile geliştirilen tek sayfa uygulaması (SPA) mimarisi üzerine inşa edilmiştir. Veriler PostgreSQL veritabanında saklanmakta; kullanıcı kimlik doğrulama işlemleri JSON Web Token (JWT) tabanlı güvenli oturum yönetimi ile sağlanmaktadır.**

Geliştirilen platform; kullanıcı kayıt ve giriş işlemleri, şifre sıfırlama e-posta akışı, kitap/film arama ve keşfetme ekranları, profil yönetimi, özel listeler oluşturma, kitap/film kütüphanesi durumu takibi (okudum/okunacak, izledim/izlenecek), içerik detay inceleme, puanlama ve yorumlama gibi temel işlevleri desteklemektedir. Backend katmanı, film verileri için TMDb; kitap verileri için Google Books ve Open Library API'leri ile entegre çalışarak içerik meta verilerini dinamik olarak içeriye almaktadır. Kullanıcı aktiviteleri (puan verme, yorum yazma, listeye ekleme vb.) sistemde ayrı bir *activity* modeli üzerinden kaydedilmekte ve takip edilen kullanıcıların aktiviteleri zaman sıralı bir sosyal akış (feed) olarak sunulmaktadır. Bu akış, Django REST Framework'ün sayfalama (pagination) mekanizmaları ve React tarafındaki "Daha Fazla Yükle" mantığıyla performanslı biçimde kullanıcıya gösterilmektedir.

Django REST Framework ve JWT tabanlı kimlik doğrulama kullanılarak tüm işlevler RESTful uç noktalar (endpoint) üzerinden sunulmuş; `ratings`, `reviews`, `library`, `custom_lists`, `social` ve `activity` gibi modüler uygulamalar (app) ile sistem mantıksal olarak parçalara ayrılmıştır. Kullanıcı profilleri, takip et/takipten çık (follow) ilişkileri, özel listeler ve aktivite beğenileri için ayrı modeller tasarlanarak çok katmanlı bir sosyal etkileşim yapısı oluşturulmuştur. API arayüzü, `drf_spectacular` ile OpenAPI/Swagger şeması üzerinden dokümante edilmiştir.

Bu rapor; projenin mimari yapısını, kullanılan teknolojileri, veritabanı tasarımını, modüler backend yapısını, React tabanlı arayüz bileşenlerini, harici API entegrasyonlarını, deneysel sonuçları ve sistemin genel değerlendirmesini detaylı biçimde sunmaktadır.

Anahtar Kelimeler — Django, React, Vite, Web uygulaması, Sosyal platform, REST API, JWT, PostgreSQL, TMDb, Google Books, Open Library.

I. Giriş

Günümüzde dijital medya tüketiminin hızlı bir şekilde artmasıyla birlikte kullanıcıların içeriklerini organize edebileceği, kişisel tercihlerine göre sınıflandırabileceği ve bu içerikler hakkında sosyal etkileşimlerde bulunabileceği çevrimiçi platformlara duyulan ihtiyaç önemli ölçüde yükselmiştir. Özellikle film ve kitap gibi kültürel içerikler üzerine kurulu sosyal topluluklar, kullanıcıların yalnızca içerik tüketmesini değil, aynı zamanda deneyimlerini paylaşarak etkileşimli bir dijital ekosisteme katılmalarını da sağlamaktadır. Bu doğrultuda geliştirilen **Web Tabanlı Sosyal Kütüphane Platformu**, kullanıcıların kitap ve film içeriklerini tek bir ortamda yönetmesini, puanlamasını, yorumlamasını ve sosyal ilişkiler kurmasını sağlayan bütünleşik bir çözüm sunmayı amaçlamaktadır.

Platform, kullanıcıların kitap ve film içeriklerini oylayabildiği, yorumlayabildiği, kişisel veya tematik özel listeler oluşturabildiği, kendi okuma/izleme durumlarını (okudum/okunacak, izledim/izlenecek) takip edebildiği ve takip ettikleri kullanıcıların aktivitelerini görebildiği bir yapıyı temel almaktadır. Kullanıcıların sistem içinde gerçekleştirdiği her eylem — bir filme puan verme, bir kitap hakkında yorum yazma, bir içeriği özel listeye ekleme, kütüphane durumunu güncelleme gibi — `activity` uygulaması üzerinden bir aktivite kaydı olarak tutulmakta ve takip ilişkileri (`social` uygulaması) ile birleştirilerek sosyal akış (feed) ekranında zaman sıralı kartlar hâlinde gösterilmektedir. Bu sayede kullanıcılar, yalnızca kendi kütüphanelerini yönetmekle kalmayıp aynı zamanda arkadaşlarının veya takip ettiği profillerin okuma/izleme alışkanlıklarını da görebilmektedir.

Gerçekleştirilen uygulama, *backend* tarafında Django ve Django REST Framework kullanılarak çok katmanlı bir mimari ile geliştirilmiştir. Kimlik doğrulama süreci, `authentication` uygulaması ve JWT tabanlı oturum yönetimi ile sağlanmakta; şifre sıfırlama gibi işlemler için e-posta üzerinden güvenli bağlantı üretilmektedir. Kullanıcıya ait profil bilgileri (`users` uygulaması), kütüphane durumları (`library` uygulaması), puanlamalar (`ratings` uygulaması), yorumlar (`reviews` uygulaması), özel listeler (`custom_lists` uygulaması) ve takip ilişkileri (`social` uygulaması) birbirinden ayrılmış modeller ve uygulamalar şeklinde tasarlanmıştır. Böylece sistem, *separation of concerns* prensibine uygun olarak modüler ve bakımı kolay bir yapıya kavuşturulmuştur.

Frontend katmanı ise React ve Vite kullanılarak tek sayfa uygulaması (SPA) şeklinde geliştirilmiştir. `react-router-dom` ile çok sayfalı bir uygulama deneyimi sağlanmış; giriş/kayıt ekranları, sosyal akış sayfası, arama ve keşfet ekranları, içerik detay sayfaları (kitap/film), kullanıcı profili ve özel liste detayları ayrı bileşenler hâlinde tasarlanmıştır. `axios` kütüphanesi ile backend API'lerine istek atılırken, JWT erişim token'ı tüm isteklerde otomatik olarak `Authorization` başlığına eklenmektedir. Sosyal akış ve profil sayfalarında sayfalama (pagination) destekli veri çekme mantığı uygulanarak hem sunucu yükü dengelenmiş hem de kullanıcıya akıcı bir gezinme deneyimi sunulmuştur.

İçerik meta verileri, film verileri için TMDb; kitap verileri için ise Google Books ve Open Library API'leri üzerinden dinamik olarak alınmaktadır. Arama ve keşfet ekranlarında kullanıcı, ilgili başlıklarla arama yaptığında backend bu harici servislerden gelen sonuçları işleyip React arayüzüne JSON formatında iletmektedir. İçerik detayına ilk girildiğinde, seçilen film veya kitap API üzerinden projeye özel veritabanına kaydedilmekte; sonraki ziyaretlerde aynı içerik doğrudan sistemin kendi veritabanından okunabilmektedir. Bu yaklaşım, hem harici servisleri gereksiz yere yormayı engellemekte hem de sık kullanılan içerikler için performans avantajı sağlamaktadır.

Bu rapor kapsamında, geliştirilen sosyal kütüphane platformunun gereksinimleri, mimari tasarımı, veritabanı yapısı, kullanılan teknolojiler, API tasarımı, kullanıcı arayüzünün bileşenleri ve deneysel sonuçlar ayrıntılı bir biçimde ele alınacaktır. Ayrıca proje sürecinde karşılaşılan teknik zorluklar ve bu zorluklara üretilen çözümler tartışılarak, ortaya çıkan sistemin yazılım mühendisliği perspektifinden değerlendirilmesi yapılacaktır.

II. Yöntem

Bu bölümde geliştirilen web tabanlı sosyal kütüphane platformunun teknik altyapısı, yazılım mimarisi, kullanılan teknolojiler, veri yönetimi stratejileri ve uygulama bileşenleri ayrıntılı biçimde ele alınmaktadır. Proje, `backend` ve `frontend` olmak üzere iki ana klasörde toplanmış; sunucu tarafında Django tabanlı bir REST API, istemci tarafında ise React ve Vite ile geliştirilen tek sayfa uygulaması (SPA) kullanılmıştır. Veriler PostgreSQL veritabanında saklanmakta, kimlik doğrulama işlemleri ise JSON Web Token (JWT) ile güvenli biçimde yönetilmektedir.

A. Genel Mimari Yaklaşım

Uygulama, istemci-sunucu tabanlı bir mimari üzerinde geliştirilmiş olup, kullanıcı tarafında çalışan React arayüzü (*frontend*) ile tüm iş mantığını, veri erişimini ve harici API entegrasyonlarını yöneten Django tabanlı REST API (*backend*) birbirinden tamamen ayrılmıştır.

Backend tarafında Django 5 ve Django REST Framework kullanılarak `api` isimli bir proje oluşturulmuş; `users`, `authentication`, `books`, `films`, `ratings`, `reviews`, `library`, `custom_lists`, `social` ve `activity` gibi modüler uygulamalar (app) üzerinden işlevler ayrıştırılmıştır. Frontend tarafında ise React bileşenleri, sayfa bazlı yapıda düzenlenmiş ve Vite ile derlenebilir hâle getirilmiştir.

Frontend ile backend arasındaki iletişim, `/api/...` altındaki RESTful uç noktalar üzerinden gerçekleşmekte; tüm veri alışverişi JSON formatında yapılmaktadır. Django REST Framework'ün sağladığı sayfalama (pagination), yetkilendirme (permissions), throttling ve şema üretimi (drf_spectacular) altyapıları aktif olarak kullanılmaktadır. CORS yapılandırması sayesinde React uygulaması ile Django API'si farklı portlarda çalışsa bile güvenli biçimde haberleşebilmektedir.

B. Frontend Yapısı ve Kullanılan Teknolojiler

Frontend tarafı `frontend/` klasörü altında, React ve Vite kullanılarak tek sayfa uygulaması (SPA) şeklinde geliştirilmiştir. Uygulamanın temel bileşenleri:

- **React Router:** `react-router-dom` kullanılarak yönlendirme yönetilmekte; `App.jsx` içinde `/login`, `/register`, `/feed`, `/search`, `/books/:id`, `/films/:id`, `/profile/:id`, `/lists/:id`, `/forgot-password` ve `/reset-password` gibi rotalar tanımlanmaktadır.
- **Axios API Katmanı:** `src/api/axios.js` dosyasında `axios` tabanlı bir istemci oluşturulmuş, `baseURL` olarak Django API'sinin kök adresi (`http://127.0.0.1:8000/api/`) tanımlanmıştır. Burada kullanılan istek kesicisi (interceptor), `localStorage` içindeki `access` token'ını `Authorization: Bearer <token>` başlığına otomatik olarak eklemektedir.
- **Korumalı Rotalar:** `ProtectedRoute.jsx` bileşeni, JWT access token'ı bulunmadığı durumda kullanıcıyı `/login` sayfasına yönlendirmekte, böylece yalnızca oturum açmış kullanıcıların feed, profil, arama vb. sayfalara erişebilmesi sağlanmaktadır.
- **Sayfa Bileşenleri:** `FeedPage`, `SearchPage`, `BookDetailPage`, `FilmDetailPage`, `ProfilePage`, `CustomListDetailPage`, `LoginPage`, `RegisterPage`, `ForgotPasswordPage` ve `ResetPasswordPage` gibi sayfalar, proje gereksinimlerinde tarif edilen fonksiyonel isterleri karşılayacak şekilde tasarlanmıştır.
- **Arayüz Bileşenleri:** `ActivityCard`, `Navbar`, `CarouselRow`, `AddToListModal`, `CreateListModal` gibi bileşenler, sosyal akıştaki aktivitelerin kart biçiminde gösterimi, arama sonuçlarının vitrin biçiminde listelenmesi ve içeriklerin özel listelere eklenmesi gibi işlevleri üstlenmektedir.
- **Stil Yönetimi:** `src/styles/` dizininde `feed.css`, `activityCard.css`, `profile.css`, `detail.css`, `auth.css`, `search.css` gibi dosyalar ile sayfa ve bileşen düzeyinde stiller tanımlanmıştır.

Bu yapı sayesinde frontend; JWT tabanlı yetkilendirme, sayfalandırılmış sosyal akış, içerik detayları, kütüphane durumları ve özel liste yönetimini kullanıcıya akıcı ve modern bir arayüz ile sunmaktadır.

C. Backend Yapısı ve API Katmanı

Backend tarafı `backend/` dizininde konumlanan Django projesi (`api`) üzerinden yürütülmektedir. `api/settings.py` dosyasında Django REST Framework (`rest_framework`), `rest_framework_simplejwt`, `corsheaders`, `drf_spectacular` gibi paketler etkinleştirilmiş; varsayılan kimlik doğrulama sınıfı olarak JWT tabanlı `JWTAuthentication` atanmıştır. Ayrıca, kullanıcı ve anonim istekler için günlük hız limitleri (throttling) tanımlanmıştır.

Backend'de her bir fonksiyonel alan ayrı bir Django uygulaması olarak tasarlanmıştır:

- **authentication:** Kayıt olma, giriş yapma, "beni hatırla" mantığıyla JWT üretimi, e-posta ile şifre sıfırlama istekleri ve şifre yenileme işlemlerini yönetmektedir. `EmailOrUsernameTokenSerializer` ile kullanıcıların hem kullanıcı adı hem de e-posta ile giriş yapabilmesi sağlanmıştır.
- **users:** `UserProfile` modeli ile profil resmi (URL veya dosya), biyografi ve profil oluşturma tarihini saklamakta; profil görüntüleme ve güncelleme uç noktalarını sağlamaktadır.
- **books** ve **films:** Kitap ve film içeriklerinin temel tablolarını tutmakta, Google Books ve TMDb servislerinden veri çekilmesini sağlayan servis fonksiyonlarını (`services.py`) içermektedir. Ayrıca içerik detay sayfasında ortalama puan, toplam oy sayısı, yorum ve liste sayısı gibi istatistikleri Django ORM ile hesaplamaktadır.
- **ratings** ve **reviews:** Kullanıcıların kitap ve filmlere verdiği puanları (`BookRating`, `FilmRating`) ve yazdıkları yorumları (`BookReview`, `FilmReview`) yönetmektedir. Puanlama modellerinde `unique_together` kısıtı ile bir kullanıcının aynı içeriğe birden fazla kez puan vermesi engellenmiştir.
- **library:** Kullanıcıların okuma/izleme durumlarını yöneten `BookLibraryStatus` ve `FilmLibraryStatus` modellerini içermektedir. "read / to_read" ve "watched / to_watch" durumları burada tutulmakta; ayrıca `LibraryStatsView` üzerinden kullanıcının kütüphane istatistikleri (kaç kitap okuduğu, kaç filmi izlediği vb.) hesaplanmaktadır.
- **custom_lists:** Kullanıcıların kendi özel listelerini (`CustomList`) ve bu listelere eklenen öğeleri (`CustomListItem`) yönetmektedir. Öğeler hem kitap hem de film olabildiği için `content_type` alanı ve ilgili yabancı anahtarlar kullanılmıştır.
- **social:** Kullanıcılar arasındaki takip ilişkilerini (`Follow` modeli) saklamakta, takip et/takipten çık işlemlerini API üzerinden sunmaktadır.
- **activity:** Kullanıcı aktivitelerini (`Activity` modeli) merkezî bir tabloda toplamaktadır. Puan verme, yorum yazma, kütüphane durumu güncelleme, listeye ekleme gibi eylemler sinyaller (signals) aracılığıyla aktivite kaydına dönüştürülmekte ve sosyal akışta kullanılmaktadır. `ActivityLike` modeli ile aktiviteler için beğeni (like) desteği de sağlanmıştır.

Tüm bu uygulamaların uç noktaları `api/urls.py` içinde `/api/users/`, `/api/auth/`, `/api/books/`, `/api/films/`, `/api/ratings/`, `/api/reviews/`, `/api/library/`, `/api/social/`, `/api/activity/` ve `/api/custom-lists/` şeklinde gruplanmıştır. Ayrıca `drf_spectacular` ile `/schema`, `/swagger` ve `/redoc` uç noktaları üzerinden otomatik API dokümantasyonu oluşturulmuştur.

D. Veritabanı Tasarımı ve Veri Yönetimi

Veritabanı `postgresql` motoru kullanılarak yapılandırılmış; `social_library` isimli bir veritabanı üzerinde ilişkisel bir model kurgulanmıştır. Temel tablolar ve ilişkiler şu şekilde özetlenebilir:

- **Kullanıcı ve Profil:** Django'nun `User` modeli temel alınmış, `UserProfile` modeli ile bire bir (one-to-one) ilişki kurulmuştur.
- **İçerik Tabloları:** `Book` ve `Film` tabloları; başlık, özet, yayın yılı, dış servis kimliği (`external_id`), poster/kapak URL'si gibi alanları tutmaktadır.
- **Puan ve Yorumlar:** `BookRating`/`FilmRating` ve `BookReview`/`FilmReview` modelleri ile kullanıcıların içeriklere bıraktığı puanlar ve yorumlar saklanmaktadır.
- **Kütüphane Durumları:** `BookLibraryStatus` ve `FilmLibraryStatus` tabloları kullanıcıların okudum/okunacak ve izledim/izlenecek durumlarını tutmaktadır.
- **Özel Listeler:** `CustomList` ve `CustomListItem` tabloları, kullanıcı tanımlı listeleri ve bu listelere eklenen kitap/film öğelerini yönetmektedir.
- **Sosyal İlişkiler ve Aktiviteler:** `Follow` tablosu takip ilişkilerini, `Activity` ve `ActivityLike` tabloları ise sosyal akışta gösterilen aktiviteleri ve bu aktivitelere verilen beğenileri saklamaktadır.

Veritabanı tasarımında bütünlük ve tutarlılık için yabancı anahtar kısıtları, benzersizlik kısıtları ve uygun indeksler kullanılmış; sık sorgulanan alanlar üzerinde performans iyileştirmeleri yapılmıştır.

E. API Entegrasyonu ve Veri Toplama Süreci

Harici API'ler ile entegrasyon, platformun içerik zenginliğini sağlayan temel bileşenlerden biridir. Kitaplar için Google Books API, filmler için ise TMDb (The Movie Database) API kullanılmaktadır.

- **Kitaplar (Google Books / Open Library):** `books/services.py` dosyasında `google_books_search` fonksiyonu ile arama sorguları Google Books API'ye iletilmekte, dönen sonuçlar filtrelenip sadeleştirilerek frontend'e uygun bir formatta geri döndürülmektedir. `import_book` fonksiyonu, seçilen bir kitabın detaylarını çekerek `Book` tablosuna kaydetmekte ve projede kullanılmak üzere kalıcı hâle getirmektedir.
- **Filmler (TMDb):** `films/services.py` dosyasında TMDb API'sine istek atan `tmdb_get` fonksiyonu ve tür eşleştirmesi yapan `TMDB_GENRE_MAP` yapısı yer almaktadır. `import_film` fonksiyonu ile film detayları çekilerek `Film` tablosuna kaydedilmekte; poster URL'si ve tür bilgileri gibi alanlar TMDb'den alınan verilerle doldurulmaktadır.

Arama sayfasında kullanıcı bir film veya kitap başlığı girdiğinde, backend bu servisler üzerinden sonuçları getirmekte; kullanıcı detay sayfasına girdiğinde ilgili içerik veritabanına kaydedilmekte ve bundan sonraki isteklerde yerel veritabanı

üzerinden hızlı erişim sağlanmaktadır. Böylece hem veri güncelliği korunmakta hem de sık erişilen içeriklerde harici API'lere bağımlılık azaltılmaktadır.

F. Mimari Yapının Değerlendirilmesi

Sistemin hem frontend hem backend tarafında modüler bir yapıda tasarlanması, geliştirme sürecinde esneklik ve yönetilebilirlik kazandırmıştır. Django tarafında farklı iş alanlarının ayrı uygulamalarda toplanması, React tarafında ise sayfa ve bileşen bazlı bir organizasyon kullanılması:

- Bakım ve hata ayıklama süreçlerini kolaylaştırmış,
- Yeni özelliklerin eklenmesini (örneğin bildirim sistemi, ek filtreler, yeni içerik türleri) daha az eforla mümkün kılmış,
- Kod tekrarını azaltmış ve yeniden kullanılabilir bileşenlerin oluşmasını sağlamış,
- Performans iyileştirmelerine ve ölçeklenebilirliğe açık bir altyapı sunmuştur.

Sonuç olarak geliştirilen teknik mimari, proje gereksinimlerini karşılayan; Django REST Framework, React, Vite, PostgreSQL, JWT ve harici API entegrasyonlarını bir araya getiren, modern ve genişletilebilir bir web uygulaması altyapısı ortaya koymaktadır.

III. Çekirdek İş Akışları ve Yalancı Kod

Bu bölümde geliştirilen sistemin temel işlevleri, gerçek koddaki akışlar sadeleştirilerek yalancı kod (pseudocode) biçiminde sunulmaktadır. Böylece Django REST Framework, React ve PostgreSQL ile gerçekleştirilen iş mantığı adım adım açıklanmaktadır.

A. Kullanıcı Kaydı (Register)

Algoritma 1: Yeni Kullanıcı Kaydı

1) İstemciden kullanıcı adı, e-posta ve parola bilgileri alınır.
2) Kullanıcı adı ve e-posta alanları kırpılır ve küçük harfe çevrilir.
3) Zorunlu alanlardan herhangi biri boş ise hata mesajı döndürülür.
4) E-posta adresi daha önce kullanılmış mı diye veritabanı sorgulanır.
5) Eğer aynı e-posta veya kullanıcı adına sahip bir kayıt varsa, "Bu e-posta veya kullanıcı adı zaten kullanımda." hatası döndürülür.
6) Şifre belirlenen kurallara göre (minimum uzunluk vb.) doğrulanır.
7) Django kullanıcı modeli kullanılarak yeni kullanıcı kaydı oluşturulur.
8) Kullanıcı profili tablosunda (`UserProfile`) varsayılan alanlarla bir profil kaydı açılır.
9) Oluşturulan kullanıcı için JWT `access` ve `refresh` token'ları üretilir.
10) Kullanıcı temel bilgileri ve token'lar JSON olarak istemciye döndürülür.

B. Kullanıcı Girişi ve JWT Üretimi

Algoritma 2: Kullanıcı Girişi

1) Girdi olarak kullanıcı adı veya e-posta (u) ve parola (p) alınır.
2) u ve p boş ise hata mesajı döndürülür ve işlem sonlandırılır.
3) Sistem önce u değerini kullanıcı adı alanında arar.
4) Eğer bulunamazsa aynı değer e-posta alanında aranır.
5) Hiçbir eşleşme bulunamazsa "Geçersiz giriş bilgileri." hatası döndürülür.
6) Kullanıcının parolası doğrulanır; yanlış ise aynı hata mesajı döndürülür.
7) Doğrulama başarılı ise Django SimpleJWT ile `access` ve `refresh` token üretilir.
8) Kullanıcı kimliği (id, kullanıcı adı, e-posta) ile birlikte JSON olarak istemciye gönderilir.

C. Şifre Sıfırlama İsteği

Algoritma 3: Şifre Sıfırlama İsteği Oluşturma

1) Girdi olarak e-posta adresi (e) alınır ve küçük harfe çevrilir.
2) e boş ise "E-posta adresi zorunludur." hatası döndürülür.
3) Veritabanında e ile eşleşen kullanıcı aranır.
4) Kullanıcı yoksa, güvenlik sebebiyle yine "Eğer böyle bir hesap varsa, şifre sıfırlama e-postası gönderildi." mesajı döndürülür.
5) Kullanıcı bulunduysa Django'nun `default_token_generator` aracıyla tek kullanımlık token üretilir.
6) Kullanıcının kimliği güvenli biçimde kodlanır (`uid`).
7) Frontend adresi kullanılarak `/reset-password/uid/token` biçiminde bir bağlantı oluşturulur.
8) Kullanıcıya bu bağlantıyı içeren e-posta gönderilir.
9) Aynı bilgilendirme mesajı istemciye döndürülür.

D. Şifre Sıfırlama Onayı

Algoritma 4: Şifre Sıfırlama Onayı

1) İstemciden `uid`, `token`, yeni parola (p_1) ve parola tekrarı (p_2) alınır.
2) p_1 ve p_2 eşleşmiyorsa hata mesajı döndürülür.
3) `uid` değeri çözülerek ilgili kullanıcı veritabanından bulunmaya çalışılır.
4) Kullanıcı bulunamazsa "Geçersiz bağlantı." hatası döndürülür.
5) Token doğrulaması başarısız ise "Geçersiz veya süresi dolmuş bağlantı." hatası döndürülür.
6) Doğrulama başarılıysa kullanıcının parolası p_1 ile güncellenir.
7) Kullanıcı kaydı kaydedilir ve "Şifre başarıyla güncellendi." mesajı döndürülür.

E. Kitap Arama ve İçe Aktarma (Google Books)

Algoritma 5: Kitap Arama

1) İstemciden arama terimi (q) alınır.

2) q boş ise hata mesajı döndürülür.
3) Google Books API için uygun bir URL oluşturulur.
4) HTTP isteği gönderilir, dönen JSON yanıtı alınır.
5) Sonuçlar arasından başlık, yazar, açıklama, sayfa sayısı ve kapak URL'si gibi alanlar seçilir.
6) Gereksiz veya eksik veriler filtrelenir.
7) İşlenmiş sonuç listesi JSON olarak frontend'e gönderilir.

Algoritma 6: Seçilen Kitabı Sisteme Kaydetme

1) İstemci, Google Books sonucundan seçilen kitabın kimliğini API'ye gönderir.
2) Backend, ilgili kimlik ile Google Books API'den detaylı kitap bilgilerini tekrar çeker.
3) Kitap daha önce veritabanına kaydedilmiş mi diye `Book` tablosu sorgulanır.
4) Varsa mevcut kayıt döndürülür, yoksa yeni bir `Book` kaydı oluşturulur.
5) Başlık, açıklama, sayfa sayısı, yazarlar ve kapak URL'si alanları doldurulur.
6) Oluşturulan `Book` nesnesinin id'si ve temel bilgileri JSON olarak istemciye döndürülür.

F. Film Arama ve İçe Aktarma (TMDb)

Algoritma 7: Film Arama

1) İstemciden arama terimi (q) alınır.
2) TMDb arama uç noktası için URL oluşturulur.
3) HTTP isteği gönderilir ve dönen JSON sonuçları alınır.
4) Her sonuç için film başlığı, özet, çıkış yılı, poster URL'si ve tür bilgileri çekilir.
5) Gereksiz alanlar atılır ve sadeleştirilmiş film listesi JSON olarak frontend'e döndürülür.

Algoritma 8: Seçilen Filmi Sisteme Kaydetme

1) İstemci, TMDb sonucundan seçilen filmin kimliğini API'ye gönderir.
2) Backend, TMDb detay uç noktasını kullanarak film bilgilerini çeker.
3) `Film` tablosu TMDb kimliğine göre sorgulanır.
4) Kayıt yoksa yeni bir `Film` nesnesi oluşturulur ve meta veriler doldurulur.
5) Film türleri TMDb tür id'lerinden yerel tür adlarına eşleştirilir.
6) Film id'si ve temel bilgiler JSON olarak istemciye döndürülür.

G. Kütüphane Durumu Güncelleme

Algoritma 9: Kitap Kütüphane Durumu

1) Girdi olarak kullanıcı (u), kitap (b) ve yeni durum (`read` veya `to_read`) alınır.
2) `BookLibraryStatus` tablosunda u ve b için bir kayıt var mı kontrol edilir.
3) Eğer kayıt yoksa yeni bir satır oluşturulur.
4) Kayıt varsa mevcut `status` alanı güncellenir.
5) Kayıt kaydedilir ve güncel durum JSON olarak döndürülür.

Algoritma 10: Film Kütüphane Durumu

1) Benzer şekilde kullanıcı (u), film (f) ve durum (`watched` veya `to_watch`) alınır.
2) `FilmLibraryStatus` tablosu kontrol edilir, yoksa kayıt oluşturulur.
3) Durum alanı güncellenir ve sonuç istemciye döndürülür.

H. Puanlama ve Yorum Ekleme

Algoritma 11: İçerik Puanlama

1) Girdi olarak kullanıcı (u), içerik (kitap veya film), ve puan değeri (r) alınır.
2) İlgili puan tablosu (`BookRating` veya `FilmRating`) kullanıcı ve içerik çifti ile sorgulanır.
3) Kayıt yoksa yeni bir puan kaydı oluşturulur.
4) Kayıt varsa mevcut puan değeri güncellenir.
5) Ortalama puan ve toplam oy istatistikleri yeniden hesaplanır.
6) Güncellenen değerler JSON olarak istemciye gönderilir.

Algoritma 12: Yorum Ekleme

1) Girdi olarak kullanıcı (u), içerik (kitap veya film) ve yorum metni (m) alınır.
2) Yorum metni boş ise hata mesajı döndürülür.
3) İlgili yorum tablosunda yeni bir satır oluşturulur ve kullanıcı ile içerik ilişkilendirilir.
4) Oluşturulan yorum, tarih ve kullanıcı bilgileri ile birlikte istemciye döndürülür.

I. Özel Liste Oluşturma ve İçerik Ekleme

Algoritma 13: Özel Liste Oluşturma

1) Girdi olarak kullanıcı (u) ve liste adı ($name$) alınır.
2) Aynı kullanıcı için aynı isimde bir liste var mı kontrol edilir.
3) Varsa hata döndürülür, yoksa yeni bir `CustomList` kaydı oluşturulur.
4) Liste id'si ve temel bilgiler JSON olarak istemciye döndürülür.

Algoritma 14: Özel Listeye İçerik Ekleme

1) Girdi olarak liste (L), içerik türü (kitap/film) ve içerik id'si alınır.
2) Aynı içerik daha önce bu listeye eklenmiş mi kontrol edilir.
3) Ekliyse hata veya uyarı döndürülür.
4) Değilse `CustomListItem` tablosuna yeni bir kayıt eklenir.
5) Gerekirse yeni bir aktivite kaydı oluşturulur (listeye ekleme olayı).

J. Takip Et / Takipten Çık

Algoritma 15: Takip Etme

1) Girdi olarak takip eden kullanıcı (u) ve takip edilecek kullanıcı (v) alınır.
2) $u = v$ ise (kullanıcı kendisini takip etmeye çalışıyorsa) hata döndürülür.
3) `Follow` tablosunda u ve v için kayıt olup olmadığı kontrol edilir.
4) Yoksa yeni bir takip kaydı oluşturulur.

5) Takip sayıları (follower/following) istatistikleri güncellenir.

Algoritma 16: Takipten Çıkma

1) Girdi olarak u ve v alınır.
2) `Follow` tablosunda u ve v ilişkisi aranır.
3) Kayıt varsa silinir, yoksa sessizce işlem sonlandırılır.

K. Aktivite Kaydı Oluşturma (Signals)

Algoritma 17: Model Olayından Aktivite Üretme

1) Bir puanlama, yorum, kütüphane durumu veya liste öğesi kaydı oluşturulduğunda ilgili sinyal tetiklenir.
2) Sinyal fonksiyonu, olayın hangi modelden geldiğini kontrol eder.
3) Modele göre aktivite türü belirlenir (örneğin `rate_book`, `review_film`, `library_update`, `custom_list_add`).
4) İlgili kullanıcı, içerik ve açıklama bilgileri hazırlanır.
5) `Activity` tablosuna yeni bir satır eklenir.
6) Gerekirse bildirim veya sayaç güncellemeleri yapılır.

L. Sosyal Akışın Backend Tarafında Oluşturulması

Algoritma 18: Akış Sorgusu

1) Girdi olarak oturum açmış kullanıcı (u) ve sayfa numarası ($page$) alınır.
2) u kullanıcısının takip ettiği kullanıcıların listesi ($followed$) elde edilir.
3) `Activity` tablosunda, kullanıcısı u veya $followed$ listesinde olan kayıtlar filtrelenir.
4) Silinmiş aktiviteler (`is_deleted = True`) hariç tutulur.
5) Kayıtlar `created_at` alanına göre azalan sırada sıralanır.
6) Django REST Framework sayfalama sınıfı ile ilgili $page$ için sonuçlar hesaplanır.
7) Seçilen aktiviteler serileştirilir ve `results`, `next`, `previous`, `count` alanlarıyla birlikte JSON olarak istemciye döndürülür.

M. Sosyal Akışın Frontend Tarafında Gösterilmesi

Algoritma 19: React ile Akışın Yüklenmesi

1) Başlangıçta `page = 1`, `activities = []` ve `hasMore = true` olarak ayarlanır.
2) Bileşen yüklendiğinde `/api/activity/feed/?page=1` isteği gönderilir.
3) Dönen `results` dizisi `activities` listesine eklenir.
4) `next` alanı boş değilse `page` bir artırılır, boşsa `hasMore = false` yapılır.
5) Kullanıcı sayfanın sonuna yaklaştığında veya "Daha Fazla Yükle" butonuna bastığında bir sonraki sayfa için aynı istek yapılır.
6) Her bir aktivite, `<ActivityCard>` bileşeni ile ekranda kart olarak gösterilir.
7) Kullanıcı beğen veya yorum butonlarına tıkladığında ilgili API istekleri tetiklenir ve kartlar gerektiğinde güncellenir.

IV. Deneysel Sonuçlar

Bu bölümde geliştirilen web tabanlı sosyal kütüphane platformunun fonksiyonel doğruluğu, performans özellikleri, hata senaryolarına verdiği tepkiler ve genel kullanıcı deneyimi ayrıntılı biçimde değerlendirilmektedir. Testler, uygulamanın gerçek kullanım senaryolarına mümkün olduğunca yakın bir ortamda çalıştırılmasıyla gerçekleştirilmiş; hem backend (Django REST API + PostgreSQL) hem de frontend (React + Vite) bileşenleri bütüncül olarak ele alınmıştır.

A. Test Ortamı ve Kurulum

Deneysel çalışmalar aşağıdaki özelliklere sahip bir geliştirme ortamında yürütülmüştür:

- İşletim Sistemi: 64 bit Linux tabanlı geliştirme ortamı,
- Sunucu Tarafı: Python 3.x, Django 5, Django REST Framework, PostgreSQL veritabanı,
- Kimlik Doğrulama: `rest_framework_simplejwt` ile JWT tabanlı oturum yönetimi,
- İstemci Tarafı: React 18, Vite tabanlı derleme zinciri, Google Chrome tarayıcısı,
- Harici Servisler: TMDb, Google Books ve Open Library API'leri,
- Çalıştırma Şekli: Backend ve frontend, yerel ortamda farklı portlar üzerinde (`localhost`) eş zamanlı olarak çalıştırılmıştır.

Deneyler sırasında veritabanında hem kitap hem de film içeren, çoklu kullanıcıya ait gerçekçi sayıda kayıt bulundurulmuştur. Özellikle sosyal akışın test edilebilmesi için farklı kullanıcı hesapları oluşturulmuş; karşılıklı takip ilişkileri tanımlanmış ve puanlama/yorum aktiviteleri simüle edilmiştir. Böylece sistemin yalnızca boş veri kümesi üzerinde değil, dolu ve ilişkili bir veri yapısı üzerinde de tutarlı çalışması sağlanmıştır.

B. Fonksiyonel Test Senaryoları

Fonksiyonel testlerde sistemin uçtan uca iş akışları manuel ve yarı otomatik olarak denetlenmiştir. Başlıca senaryolar şunlardır:

- **Kimlik Doğrulama Akışları:**
  - Yeni kullanıcı kaydı oluşturma,
  - Kullanıcı adı veya e-posta ile giriş yapma,
  - JWT token üretimi ve yenileme,
  - Korumalı sayfalara (profil, sosyal akış, kütüphane, özel listeler) yalnızca giriş yapmış kullanıcıların erişebilmesi.
- **Şifre Sıfırlama:**
  - Kayıtlı bir e-posta ile şifre sıfırlama isteği gönderme,
  - Geçersiz veya kayıtlı olmayan e-posta adresi ile istek gönderme,
  - E-posta ile gelen bağlantı üzerinden yeni şifre belirleme,
  - Süresi geçmiş veya bozulmuş token ile deneme yapılması.
- **İçerik Yönetimi:**

  - Kitap arama, detayını görüntüleme ve veritabanına içe aktarma,
  - Film arama, TMDb üzerinden detay çekme ve sisteme kaydetme,
  - İçerik detay sayfalarında meta verilerin (başlık, özet, yıl, türler, kapak) tutarlı biçimde gösterilmesi.
- **Kütüphane ve Etkileşim:**
  - Kitaplar için "Okudum / Okunacak", filmler için "İzledim / İzlenecek" durumlarının güncellenmesi,
  - Aynı içeriğe birden fazla kez puan vermeye çalışılması durumunda mevcut puanın güncellenmesi,
  - Yorum ekleme, düzenleme ve silme işlemlerinin yalnızca yorumu oluşturan kullanıcı tarafından yapılabilmesi.
- **Sosyal Özellikler:**
  - Kullanıcı profillerini görüntüleme ve profil düzenleme,
  - Takip et / takipten çık işlevleri ve takipçi sayılarının güncellenmesi,
  - Özel liste oluşturma, listeye kitap/film ekleme, listeleri profilde görüntüleme,
  - Aktivite beğenme (like) ve akışta beğeni sayısının güncellenmesi.

Bu senaryoların her birinde, beklenen veritabanı kayıtlarının oluşturulduğu, yanlış kullanıcı veya içerik erişimlerinde uygun hata kodlarının (`400`, `401`, `403`, `404`) döndürüldüğü ve React arayüzünde geri bildirim mesajlarının doğru şekilde gösterildiği doğrulanmıştır. Özellikle JWT tabanlı kimlik doğrulama mekanizmasının, yetkisiz istekleri tutarlı biçimde engellediği ve oturumu olmayan kullanıcıları giriş sayfasına yönlendirdiği gözlemlenmiştir.

C. Performans Gözlemleri ve API Tepki Süreleri

Performans değerlendirmesinde hem backend API tepki süreleri hem de frontend tarafındaki sayfa yükleme deneyimi gözlemlenmiştir. Yerel geliştirme ortamında yapılan ölçümlerde:

- Giriş yapmış bir kullanıcı için sosyal akışın ilk sayfasının yüklenme süresi, tipik bir senaryoda birkaç yüz milisaniye mertebesinde kalmıştır.
- Kitap ve film arama isteklerinde, harici servislere yapılan HTTP çağrıları nedeniyle gecikmeler görülse de, sonuçlar kullanıcı açısından kabul edilebilir sürelerde (genellikle 1–2 saniye aralığında) dönmüştür.
- Sayfalama (pagination) kullanılması sayesinde, sosyal akışta her istekte yalnızca 10–15 aktivite çekilmiş ve böylece hem sunucu yükü hem de istemci tarafındaki render maliyeti azaltılmıştır.
- Kütüphane durumu, puan ve yorum güncellemelerinde sadece ilgili kayıt güncellendiğinden, bu isteklerin tepki süreleri çoğunlukla çok kısa (100–300 ms aralığında) kalmıştır.

Ayrıca, veritabanı tarafında sık kullanılan alanlara indeks eklenmesi (örneğin kullanıcı-id, içerik-id, oluşturulma tarihi gibi) sorgu sürelerini olumlu yönde etkilemiş; sosyal akış ve profil ekranlarında yoğun sorgu yapılan senaryolarda dahi belirgin bir yavaşlama gözlemlenmemiştir.

D. Hata Senaryoları ve Dayanıklılık

Sistemin kararlılığını değerlendirmek amacıyla çeşitli hata ve sınır durumları da bilinçli olarak tetiklenmiştir. Öne çıkan senaryolar:

- Eksik veya hatalı form girişleri (boş alanlar, geçersiz e-posta formatı, kısa şifre),
- Yetkisiz kullanıcıların korumalı uç noktalara istek atması,
- Geçersiz veya süresi dolmuş şifre sıfırlama bağlantıları,
- Harici API'lerden hata dönmesi veya geçici erişim sorunları,
- Aynı içeriğe tekrar tekrar liste ekleme veya puan verme denemeleri.

Bu durumlarda API katmanında uygun hata cevaplarının döndürüldüğü, React arayüzünde kullanıcıya anlamlı ve açıklayıcı hata mesajları gösterildiği görülmüştür. Özellikle harici API çağrılarında ağ hatası veya beklenmeyen yanıt alındığında, sistemin tamamen çökmesi engellenmiş; bunun yerine kullanıcıya tekrar deneme veya daha sonra tekrar deneme öneren mesajlar sunulmuştur. Veritabanı düzeyindeki eşsizlik kısıtları sayesinde, aynı kullanıcının aynı içeriğe birden fazla puan kaydı oluşturması engellenmiş; yarış durumlarında veri tutarlılığı korunmuştur.

E. Kullanıcı Deneyimi ve Arayüz Geri Bildirimleri

React tabanlı arayüzde form doğrulama, buton durumları, yükleniyor (loading) göstergeleri ve hata/başarı bildirimleri test edilmiştir. Yanlış parola veya kayıtlı olmayan e-posta ile giriş denemelerinde, kullanıcıya anlaşılır hata mesajları gösterildiği; zorunlu alanlar boş bırakıldığında ilgili alanların görsel olarak vurgulandığı gözlemlenmiştir.

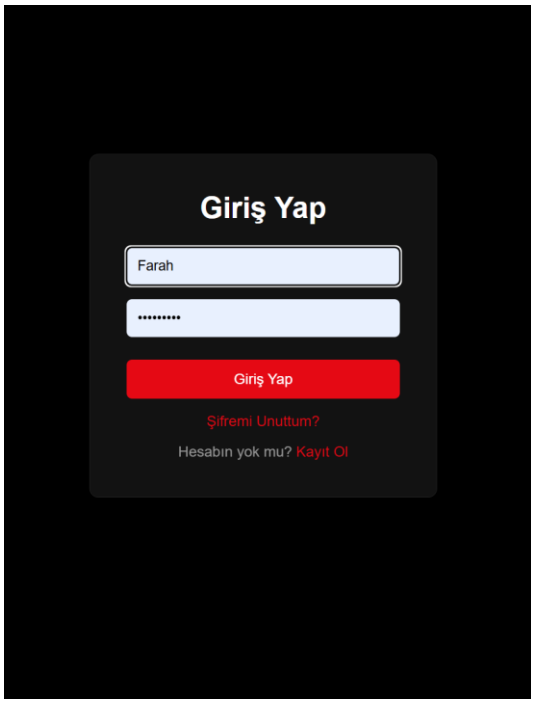
Uzun sürebilecek işlemlerde (özellikle harici API aramaları) butonlarda yükleniyor animasyonları kullanılmış, işlem tamamlandığında kısa süreli başarı mesajları ile kullanıcı bilgilendirilmiştir. Uzun süreli kullanım senaryolarında (art arda arama, profil geçişleri, liste işlemleri vb.) uygulamanın kararlı şekilde çalıştığı ve oturum bilgilerinin doğru yönetildiği görülmüştür.

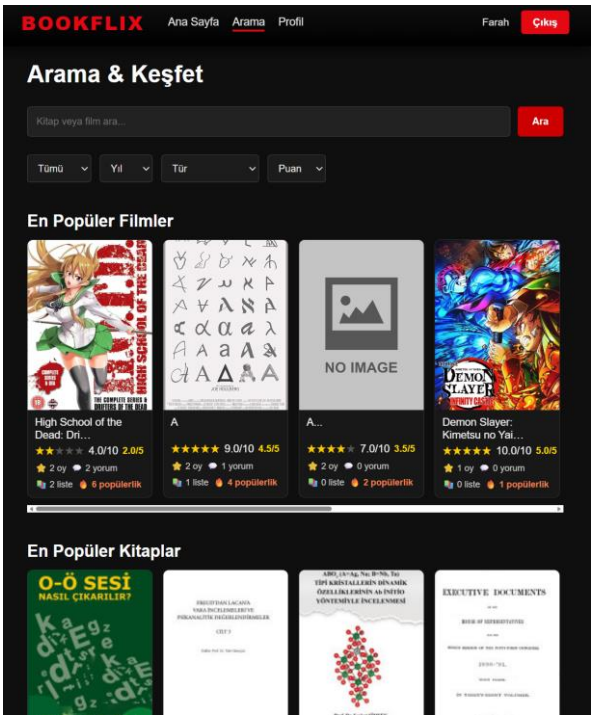
F. Genel Değerlendirme

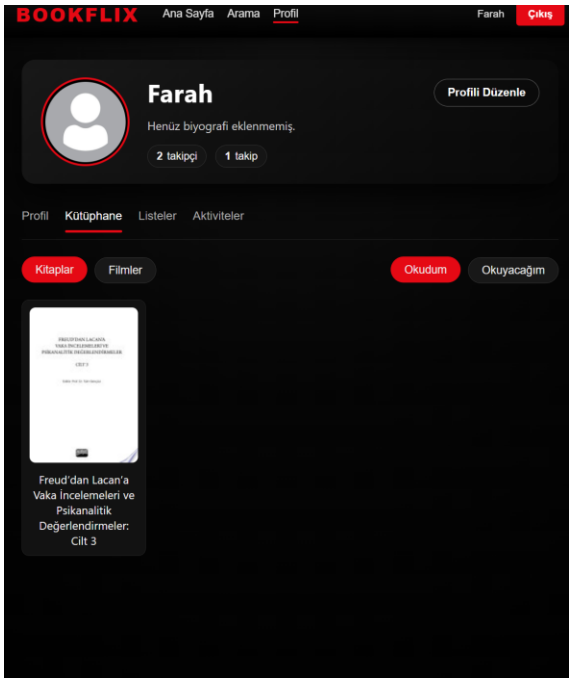
Deneysel sonuçlar, geliştirilen platformun temel fonksiyonlar açısından doğru ve tutarlı çalıştığını, JWT tabanlı kimlik doğrulama ile güvenli bir yapı sunduğunu ve sosyal akış, arama, kütüphane yönetimi gibi etkileşimli ekranlarda yeterli performans sağladığını göstermektedir. Hata durumlarında kullanıcıyı doğru yönlendiren mesajlar üretilmesi ve sistemin kararlı kalması, genel kullanıcı deneyimini güçlendirmiştir.

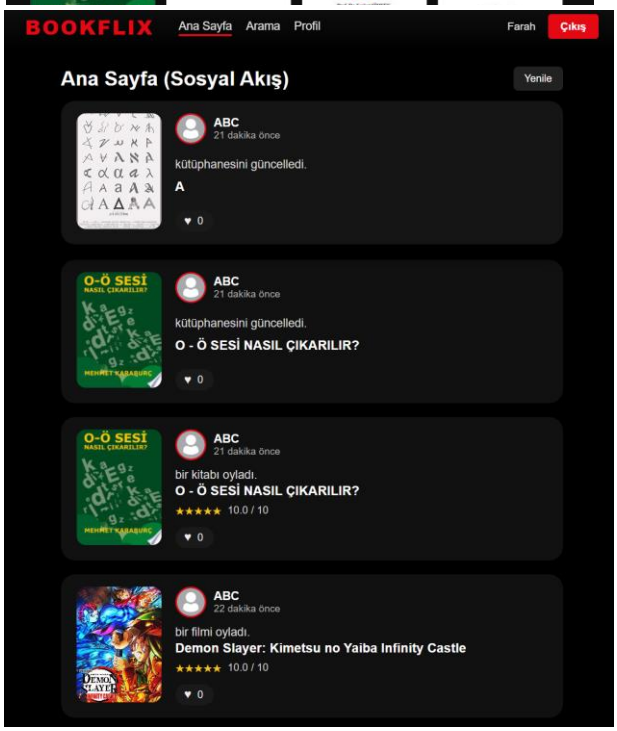
Modüler backend yapısı (Django uygulamaları) ve bileşen tabanlı React arayüzü sayesinde proje, yeni özelliklerin eklenmesine ve gelecekte yapılacak geliştirmelere uygun, genişletilebilir bir altyapı sunmaktadır.

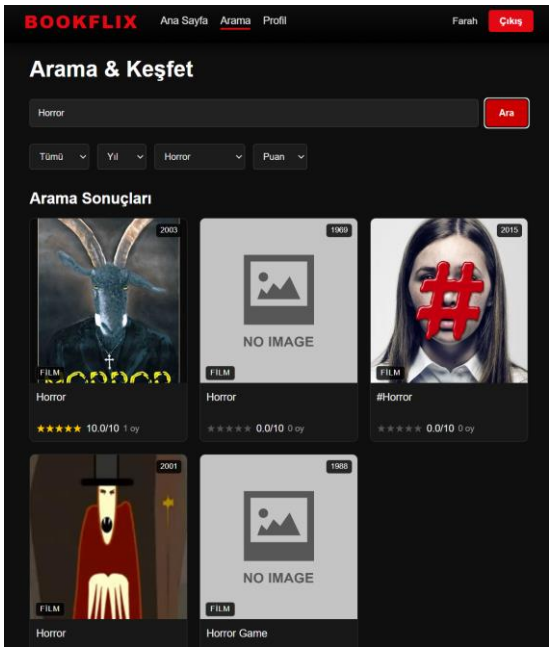
Web Uygulaması Ekran Görüntüleri:

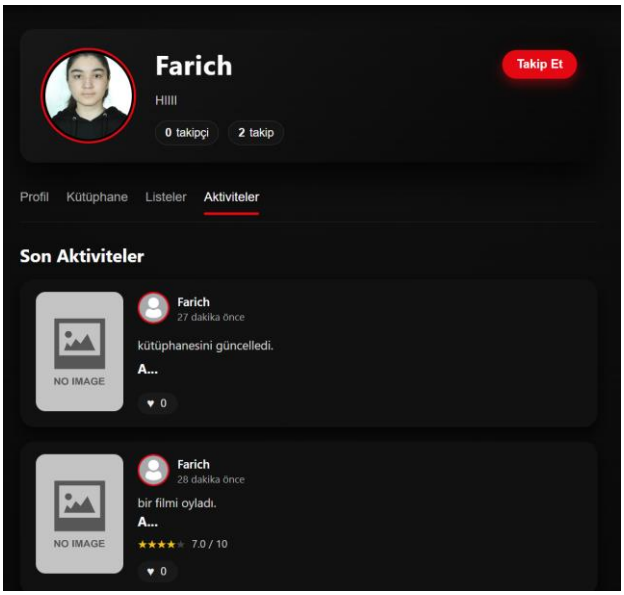




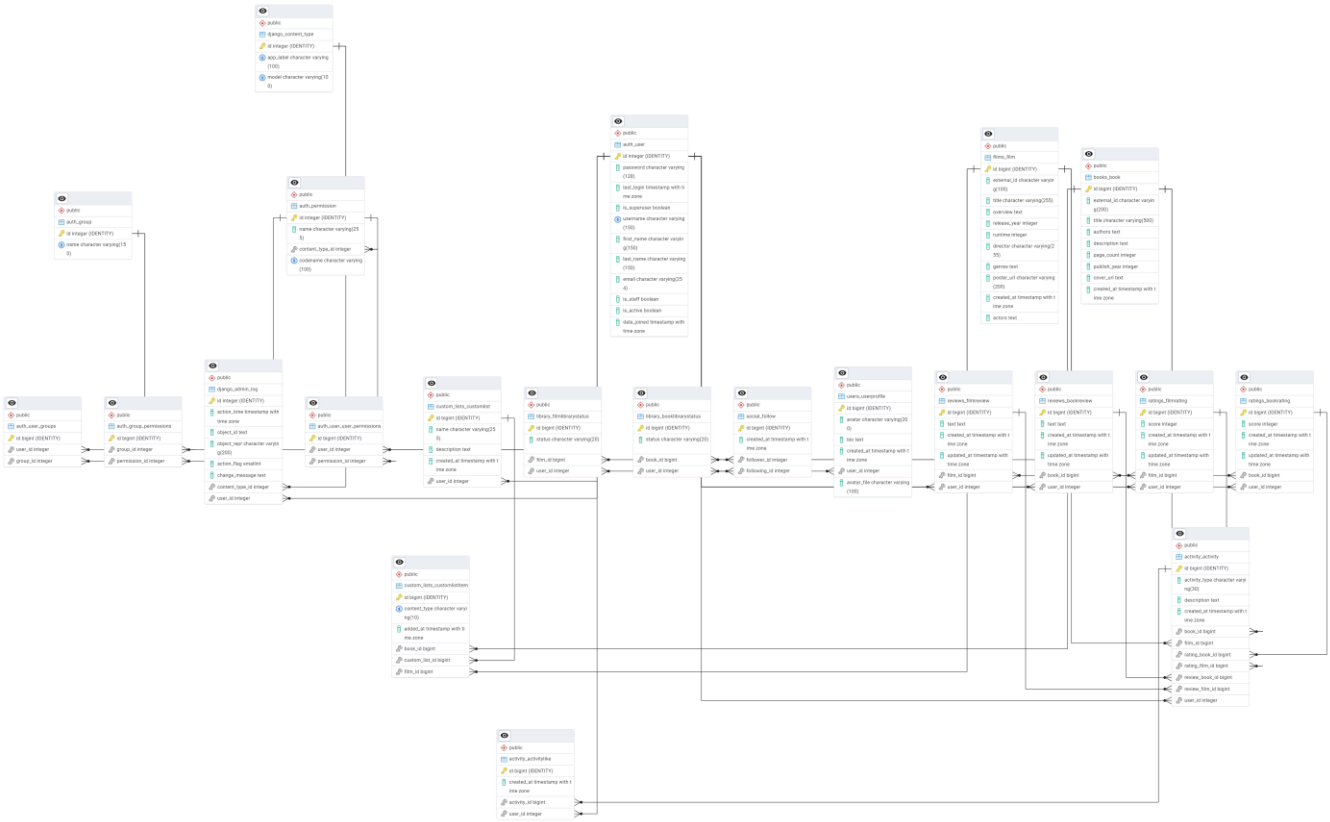


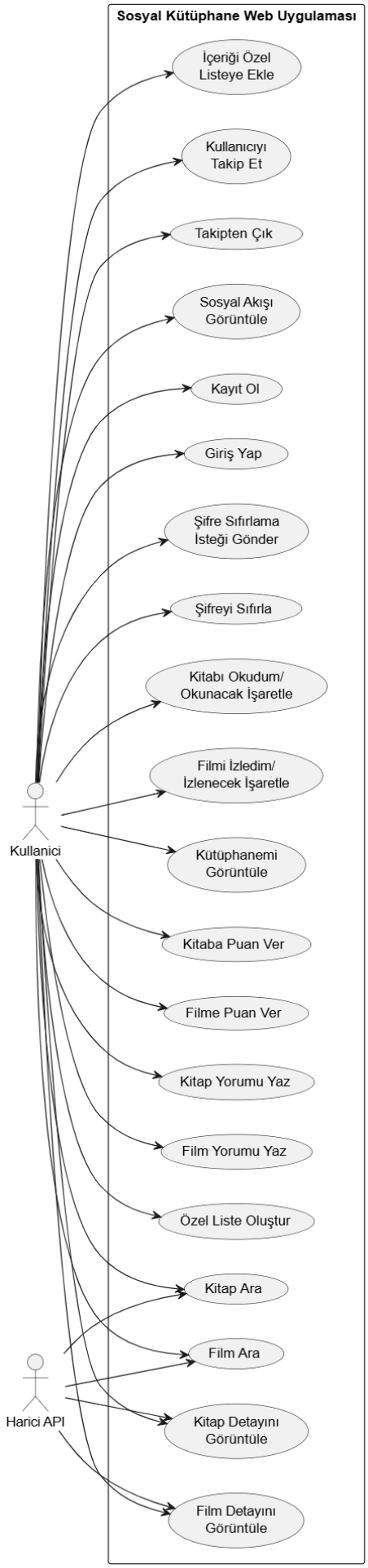






Class Diyagramı:





Use Case Diyagramı:

V. Sonuç

Bu proje kapsamında, kullanıcıların kitap ve film içeriklerini tek bir platform üzerinde yönetebildiği, bu içerikler hakkında puanlama ve yorum yapabildiği, özel listeler oluşturabildiği ve diğer kullanıcıların aktivitelerini takip edebildiği web tabanlı bir *sosyal kütüphane* uygulaması geliştirilmiştir. Sistem, Django ve Django REST Framework tabanlı bir backend ile React ve Vite tabanlı bir frontend katmanından oluşmakta; PostgreSQL veritabanı ve JWT tabanlı kimlik doğrulama mekanizması sayesinde güvenli, ölçeklenebilir ve modüler bir altyapı sunmaktadır.

Gerçekleştirilen çalışma ile yalnızca statik bir içerik gösterim sistemi kurulmamış, aksine kullanıcıların kendi kütüphanelerini aktif olarak yönetebildiği, içerikleri okundu/okunacak veya izlendi/izlenecek şeklinde etiketleyebildiği, özel listeler oluşturabildiği ve sosyal akış üzerinden başkalarının aktivitelerini görebildiği etkileşimli bir sosyal ortam ortaya konmuştur. Kullanıcılar; kitap ve filmler için puan verebilmekte, yorum yazabilmekte, bu yorum ve puanlamalar üzerinden içeriklerin genel istatistiklerini gözlemleyebilmektedir. Bu sayede sistem, klasik bir veritabanı uygulamasından ziyade, kullanıcı davranışlarının ve tercihlerinin de ön planda olduğu bir sosyal platform işlevi görmektedir.

Projenin önemli katkılarından biri, içerik bilgilerinin manuel veri girişiyle değil, harici API servisleri üzerinden dinamik olarak elde edilmesidir. Filmler için TMDb, kitaplar için Google Books ve Open Library API'lerinin kullanılması sayesinde, içerik meta verileri (başlık, özet, yıl, türler, yazarlar/yönetmenler, kapak görselleri vb.) gerçek zamanlı ve güncel bir şekilde sisteme aktarılmaktadır. Bu yaklaşım, hem veri giriş yükünü ortadan kaldırmakta hem de platformun ölçeklenebilirliğini ve sürdürülebilirliğini artırmaktadır. Gerekli durumlarda içerikler veritabanına kaydedilerek sonraki erişimlerde daha hızlı yanıt alınması sağlanmıştır.

Sistem tasarımı sürecinde, veri modelinin sosyal ilişkileri destekleyecek biçimde kurgulanmasına özel önem verilmiştir. Kullanıcı, içerik (kitap/film), puanlama, yorum, kütüphane durumu, özel listeler, takip ilişkileri ve aktiviteler için ayrı tabloların ve uygulama modüllerinin kullanılması; hem kodun bakımını kolaylaştırmış hem de iş mantığının modüler ve anlaşılır olmasını sağlamıştır. Özellikle `activity` uygulaması sayesinde; puan verme, yorum yazma, kütüphane durumu güncelleme ve listeye ekleme gibi eylemler tek bir merkezî yapı üzerinden takip edilebilmiş ve sosyal akış ekranında zaman sıralı olarak gösterilebilmiştir.

Deneysel sonuçlar ve testler, uygulamanın temel fonksiyonlar açısından beklenen gereksinimleri karşıladığını göstermiştir. Kullanıcı kaydı, giriş, şifre sıfırlama, içerik arama, içerik detay görüntüleme, kütüphane yönetimi, puanlama, yorum ekleme, özel liste yönetimi, takip etme ve sosyal akış gibi senaryolarda hem backend hem de frontend tarafında tutarlı davranışlar gözlemlenmiştir. JWT tabanlı kimlik doğrulama sayesinde yetkisiz erişimler engellenmiş; korumalı uç noktalara yalnızca giriş yapmış kullanıcıların erişebilmesi sağlanmıştır. Sosyal akışın sayfalama (pagination) ile sunulması performans açısından olumlu sonuç vermiş, yoğun veri içeren senaryolarda bile kullanıcıya akıcı bir gezinme deneyimi sunulmuştur.

Bu proje aynı zamanda modern web uygulaması geliştirme kültürüne dair önemli deneyimler kazandırmıştır. Django REST Framework ile API tasarımı, JWT ile kimlik doğrulama, PostgreSQL veritabanı modelleme ve React ile tek sayfa uygulaması (SPA) geliştirme adımları; yazılım mühendisliği açısından katmanlı mimari, *separation of concerns*, modülerlik ve yeniden kullanılabilirlik gibi temel prensiplerle birlikte ele alınmıştır. Harici API entegrasyonları, hata yönetimi, form doğrulama, kullanıcıya geribildirim verme ve sosyal etkileşim özellikleri, projenin gerçek hayatta kullanılabilir bir ürün olmasına yönelik önemli yapı taşlarını oluşturmuştur.

Gelecek çalışmalar kapsamında sistemin daha da geliştirilebilmesi için aşağıdaki yönelimler öne çıkmaktadır:

- Platformun Android ve iOS için yerel veya hibrit bir mobil uygulama sürümünün geliştirilmesi,
- Gerçek zamanlı bildirim (push notification veya WebSocket tabanlı) modüllerinin eklenerek yeni yorum, beğeni, takip veya liste ekleme gibi olayların anlık olarak kullanıcıya iletilmesi,
- Kullanıcıların geçmiş davranışlarına (puanlar, listeler, kütüphane durumları, takip ilişkileri vb.) dayalı gelişmiş bir öneri sistemi (recommendation engine) tasarlanması ve entegre edilmesi,
- Etkileşim ve sosyal ağ analizleri ile kullanıcılar arasındaki benzerliklerin, öne çıkan içeriklerin ve topluluk yapıların incelenmesi ve bu analizlerin arayüze yansıtılması,
- Ölçeklenebilirlik ve dağıtık mimari bağlamında, sistemin konteyner tabanlı (Docker) veya bulut altyapısına taşınarak daha geniş kullanıcı kitlelerine hizmet verebilir hâle getirilmesi.

Sonuç olarak bu çalışma, hem teknik açıdan güncel teknolojileri bir araya getiren tam yığın (full-stack) bir web uygulaması ortaya koymuş, hem de kitap ve film içerikleri etrafında şekillenen sosyal etkileşimleri destekleyen bütünleşik bir platform tasarlamıştır. Yazılım mimarisi, veri modeli ve kullanıcı arayüzü birlikte değerlendirildiğinde, projenin hem akademik hem de pratik açıdan değerli bir deneyim sunduğu ve ileride geliştirilebilecek daha kapsamlı sistemler için sağlam bir temel oluşturduğu söylenebilir.

Kaynaklar

[1] The Movie Database (TMDb), *TMDb API Documentation*. Erişim: 2025. Çevrimiçi: https://developer.themoviedb.org

[2] Google, *Google Books API Documentation*. Erişim: 2025. Çevrimiçi: https://developers.google.com/books

[3] Open Library, *Open Library Developer API*. Erişim: 2025. Çevrimiçi: https://openlibrary.org/developers/api

[4] IEEE, *IEEE Conference Paper Template and Guidelines*, 2024.

[5] Kocaeli Üniversitesi, *Bilgisayar Mühendisliği Bölümü Yönergeleri*, 2024.

[6] Django Software Foundation, *Django 5.0 Documentation*. Erişim: 2025. Çevrimiçi: https://docs.djangoproject.com

[7] T. Christie ve diğ., *Django REST Framework Official Documentation*. Erişim: 2025. Çevrimiçi: https://www.django-rest-framework.org

[8] J. S. Watt, *djangorestframework-simplejwt Documentation*. Erişim: 2025. Çevrimiçi: https://django-rest-framework-simplejwt.readthedocs.io

[9] React, *React Official Documentation*. Erişim: 2025. Çevrimiçi: https://react.dev
[10] Vite, *Vite – Next Generation Frontend Tooling*. Erişim: 2025. Çevrimiçi: https://vitejs.dev
[11] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*. Erişim: 2025. Çevrimiçi: https://www.postgresql.org/docs
[12] S. Schaffer, *Django REST Framework API Tutorial (Playlist)*. YouTube video serisi, 2023. Erişim: 2025. Çevrimiçi: https://www.youtube.com
[13] Web Dev Simplified, *React Router Tutorial – Full Course*. YouTube videosu, 2023. Erişim: 2025. Çevrimiçi: https://www.youtube.com